

ARQUITECTURA DE SOFTWARE

1. DEFINICIÓN

La arquitectura de software es la organización fundamental de un sistema, que define:

- Componentes (subsistemas)
- Relaciones entre ellos
- Principios de diseño
- Restricciones

Es una decisión estratégica, no técnica solamente.

OBJETIVOS

- Escalabilidad (crecer sin romper)
- Mantenibilidad (fácil de modificar)
- Rendimiento
- Seguridad

CONCEPTOS CLAVE

Componente

Parte del sistema con responsabilidad clara

Ej: módulo de usuarios

Conector

Forma en que se comunican

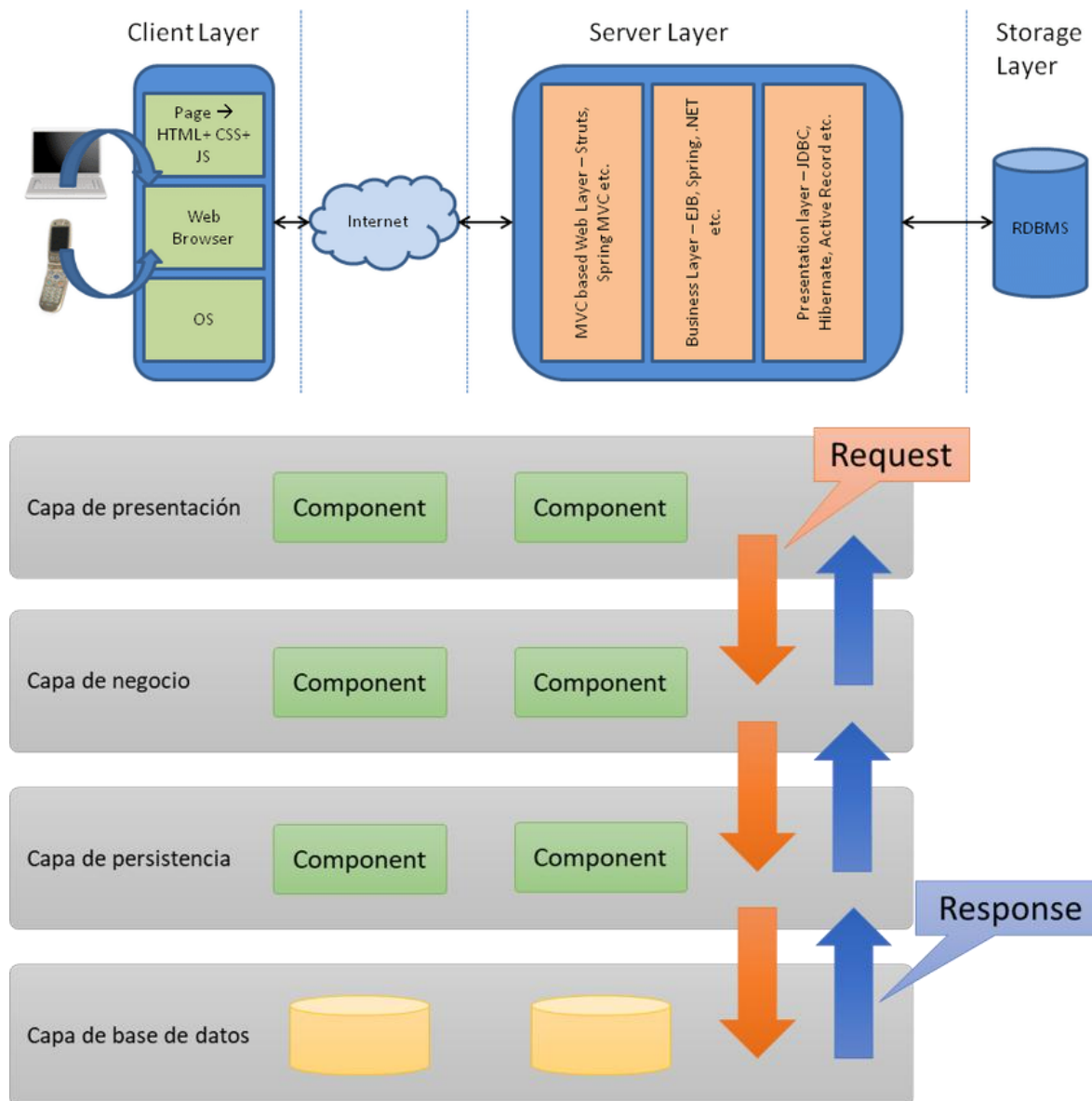
Ej: API REST

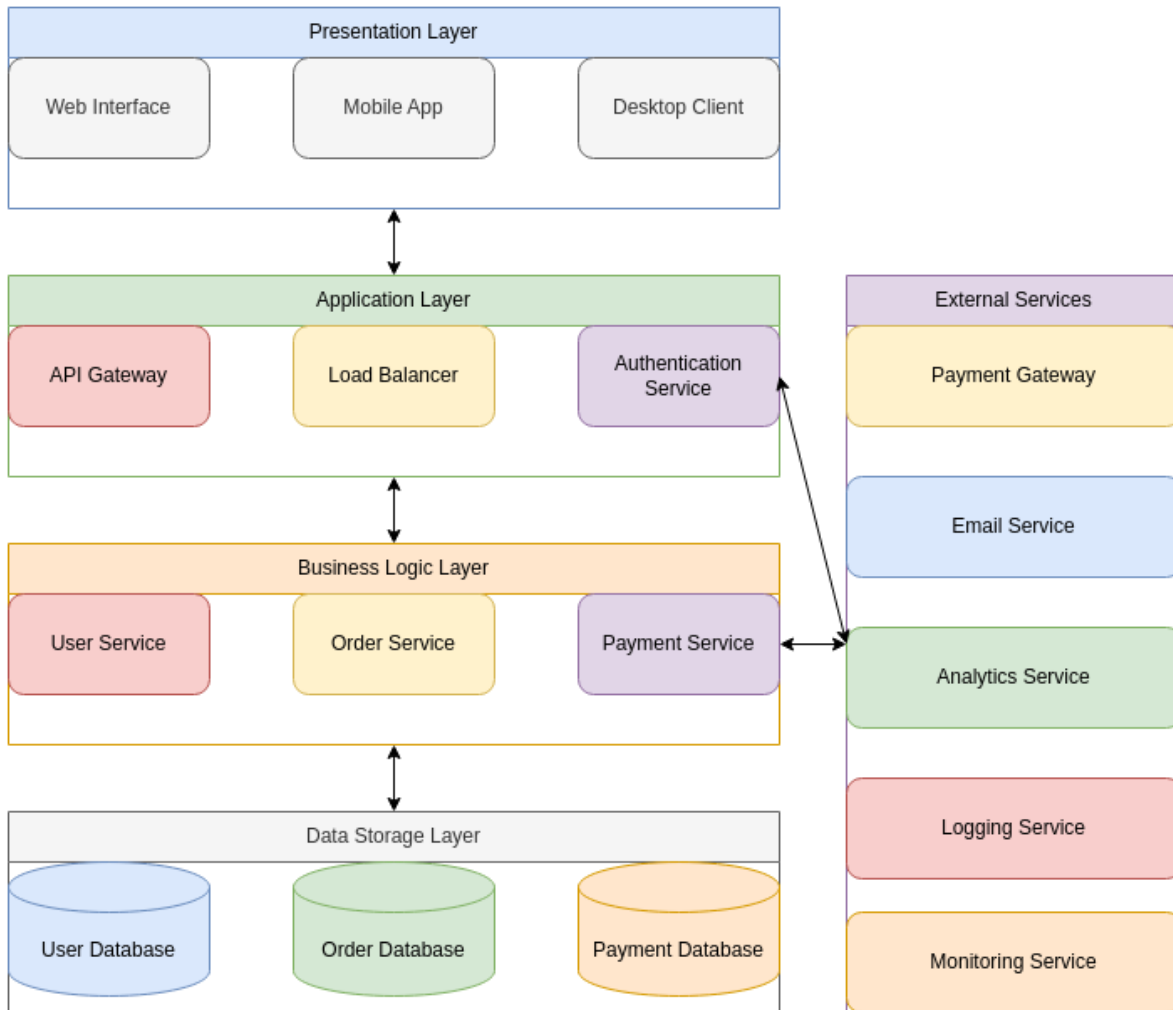
Estilo arquitectónico

Patrón general (capas, microservicios, etc.)

2. TIPOS DE ARQUITECTURA (PROFUNDO + EJEMPLOS)

2.1 Arquitectura en Capas (Layered Architecture)





TEORÍA

Divide el sistema en niveles jerárquicos donde:

- Cada capa tiene una responsabilidad
- Cada capa solo interactúa con la siguiente

Capas reales:

1. Presentación (UI)
2. Aplicación / negocio
3. Persistencia (datos)

FLUJO REAL

Usuario → UI → lógica → base de datos → respuesta

ONG (capacitaciones)

- UI → trabajadores ven cursos
- Lógica → validación de inscripción
- Datos → cursos y usuarios

Ejemplo concreto:

“Si el curso está lleno → no permite inscripción”

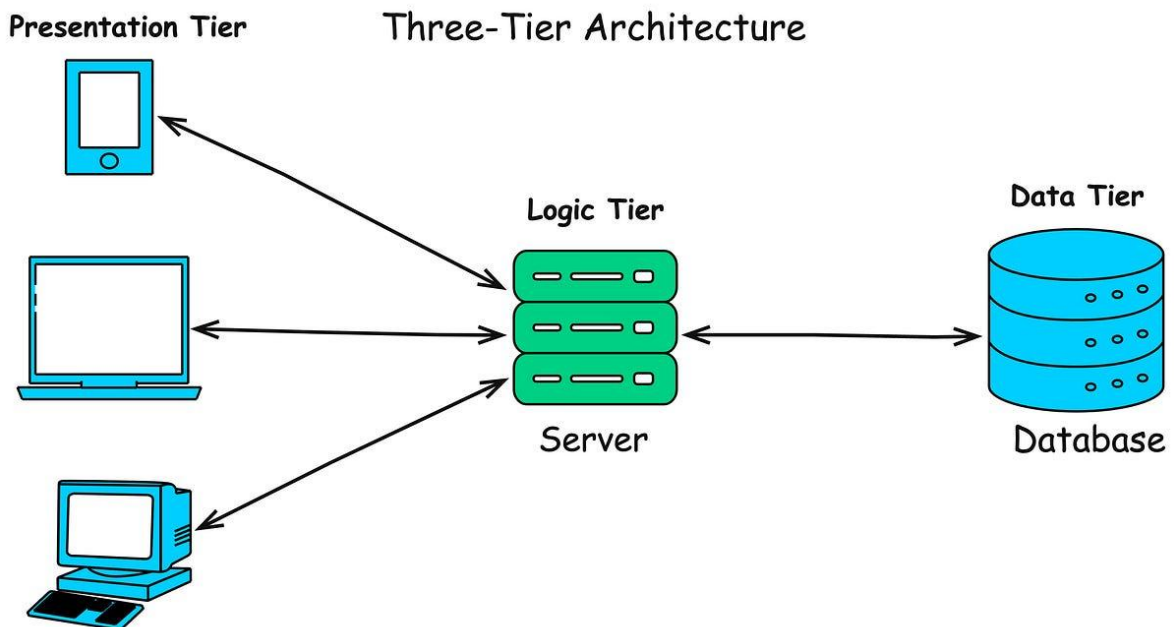
LinkedIn escolar

- UI → perfil estudiante
- Lógica → ranking de habilidades
- Datos → competencias

PROBLEMA REAL

- Si rompes capas → caos
Ej: lógica en frontend

2.2 Cliente – Servidor





TEORÍA

Sistema distribuido donde:

- Cliente solicita
- Servidor responde

PROCESO REAL

1. Usuario hace clic
2. Navegador envía request
3. Servidor procesa
4. BD responde
5. Servidor devuelve datos

ONG

Trabajador:

“Ver cursos”

→ servidor responde lista

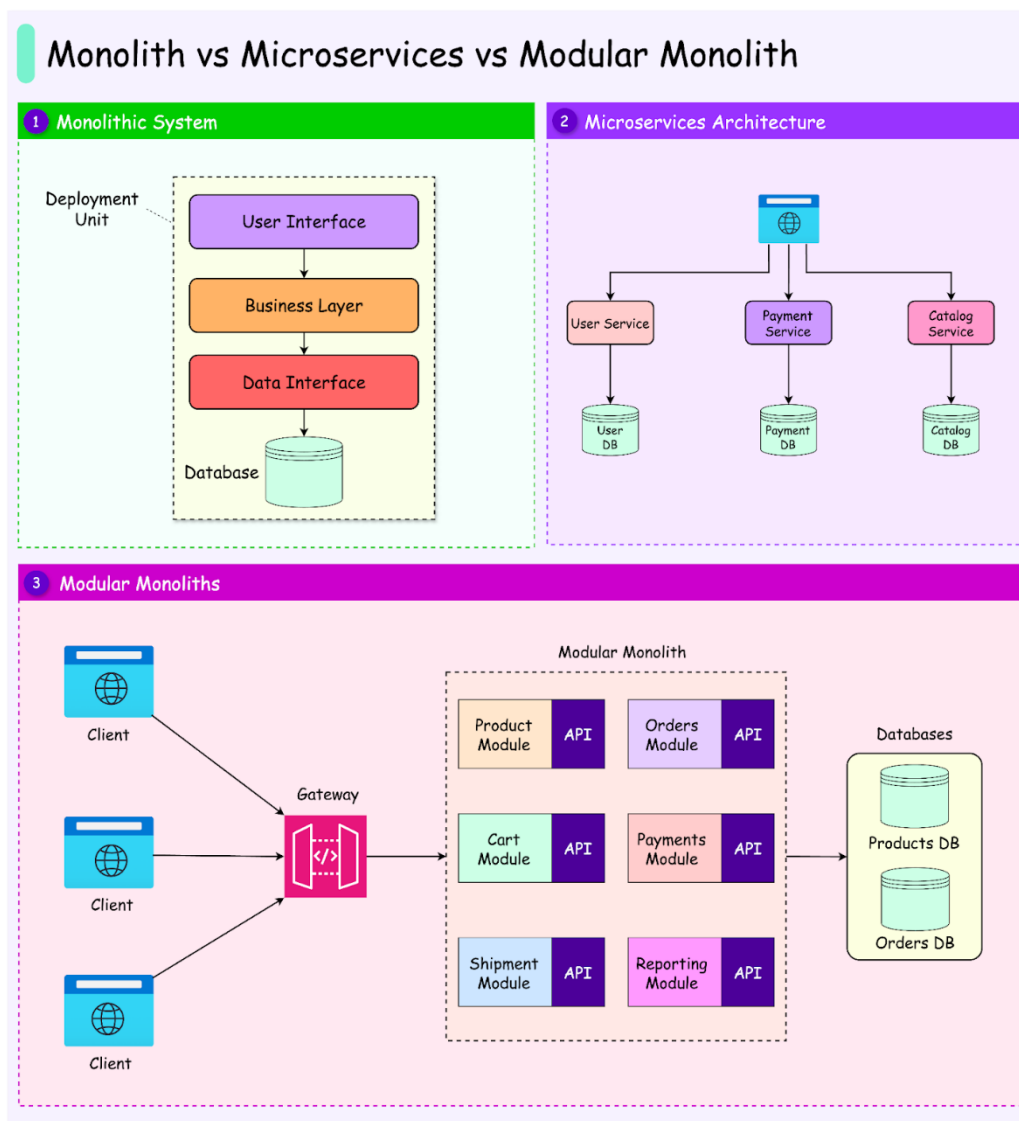
LinkedIn

Estudiante:

“Ver perfil”

→ servidor devuelve datos

2.3 Monolítica



TEORÍA

Todo el sistema está en una sola aplicación

ONG

Un solo backend maneja:

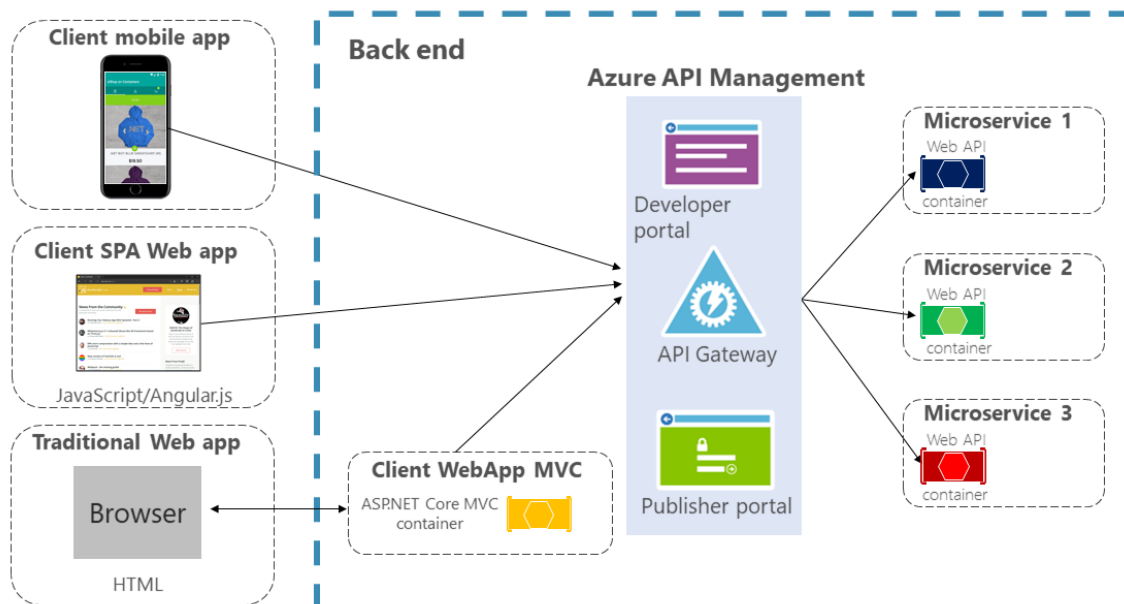
- usuarios
- cursos
- reportes

PROBLEMAS REALES

- Un error → tumba todo
- Difícil escalar

2.4 Microservicios

API Gateway with Azure API Management Architecture



TEORÍA

Sistema dividido en servicios independientes

ONG

- servicio usuarios
- servicio cursos
- servicio certificados

LinkedIn

- perfiles
- habilidades
- recomendaciones

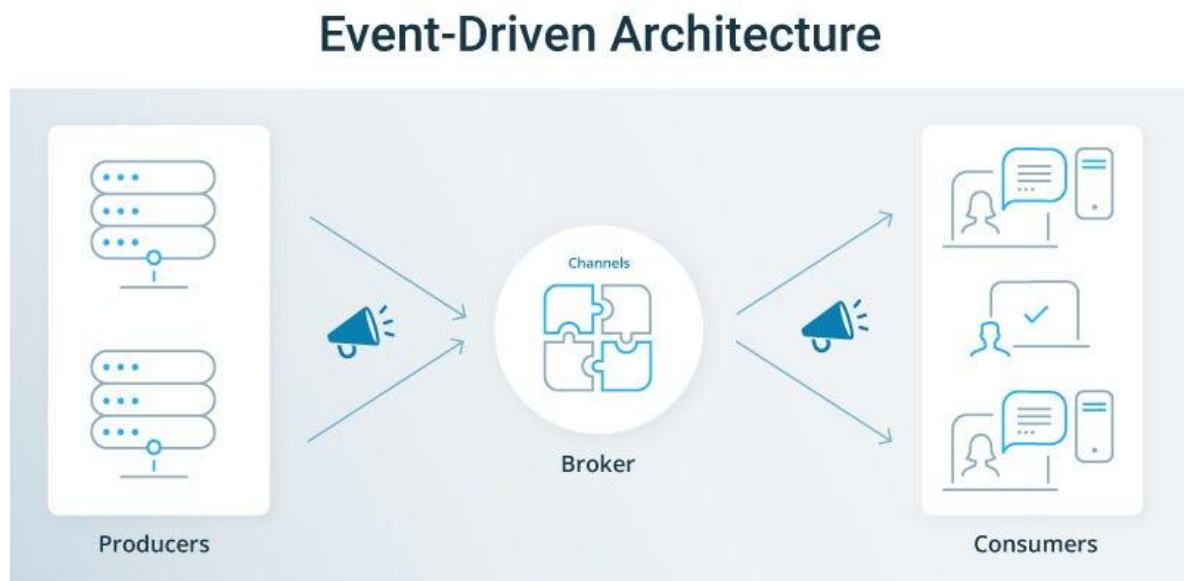
PROBLEMAS

- Complejo
- Difícil de implementar

IMPORTANTE:

NO lo uses en cursos básicos

2.5 Event-Driven Architecture



TEORÍA

Los componentes se comunican mediante eventos

ONG

Evento:

“usuario se inscribe”

→ sistema envía correo

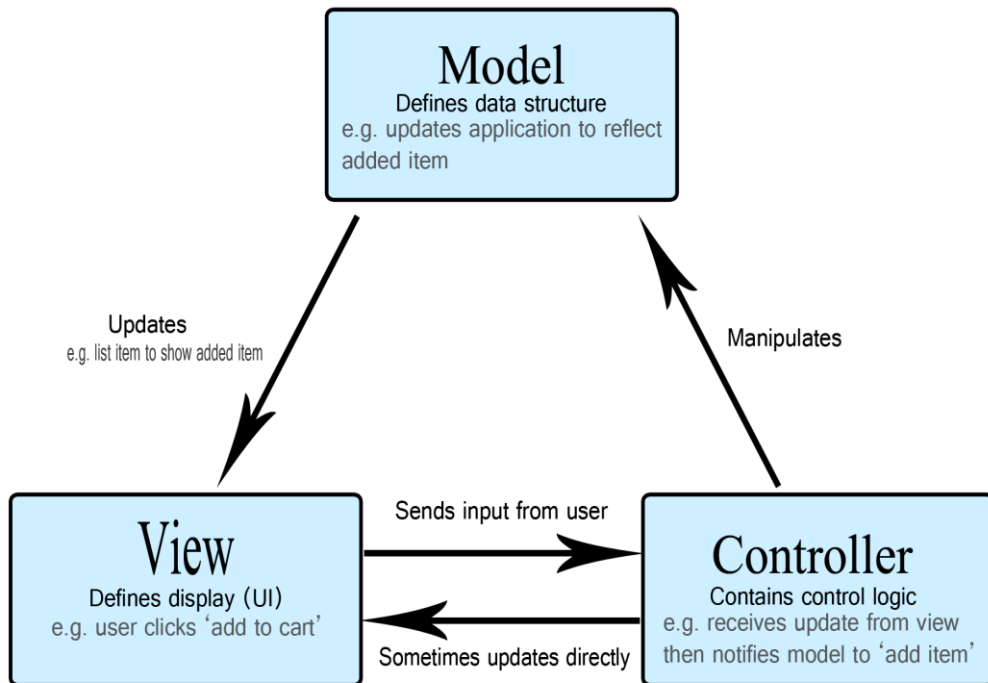
LinkedIn

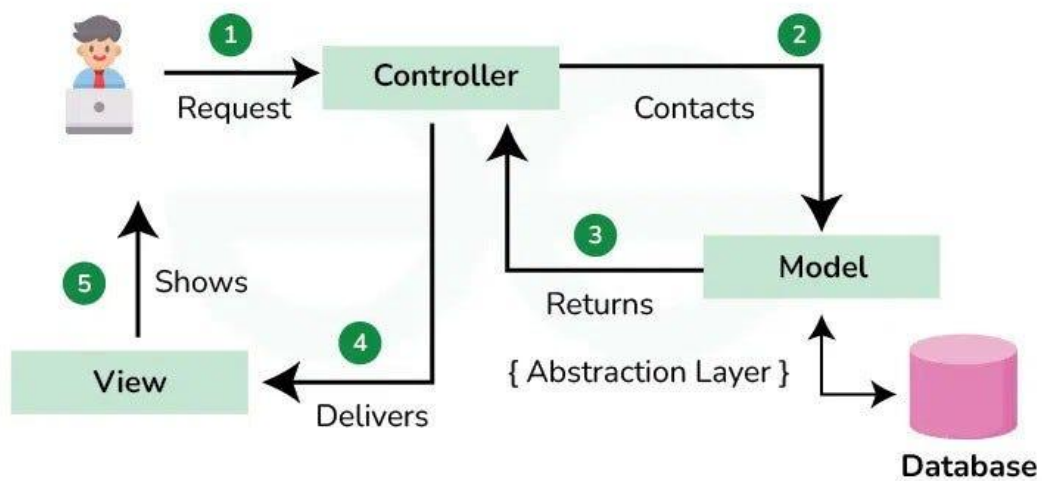
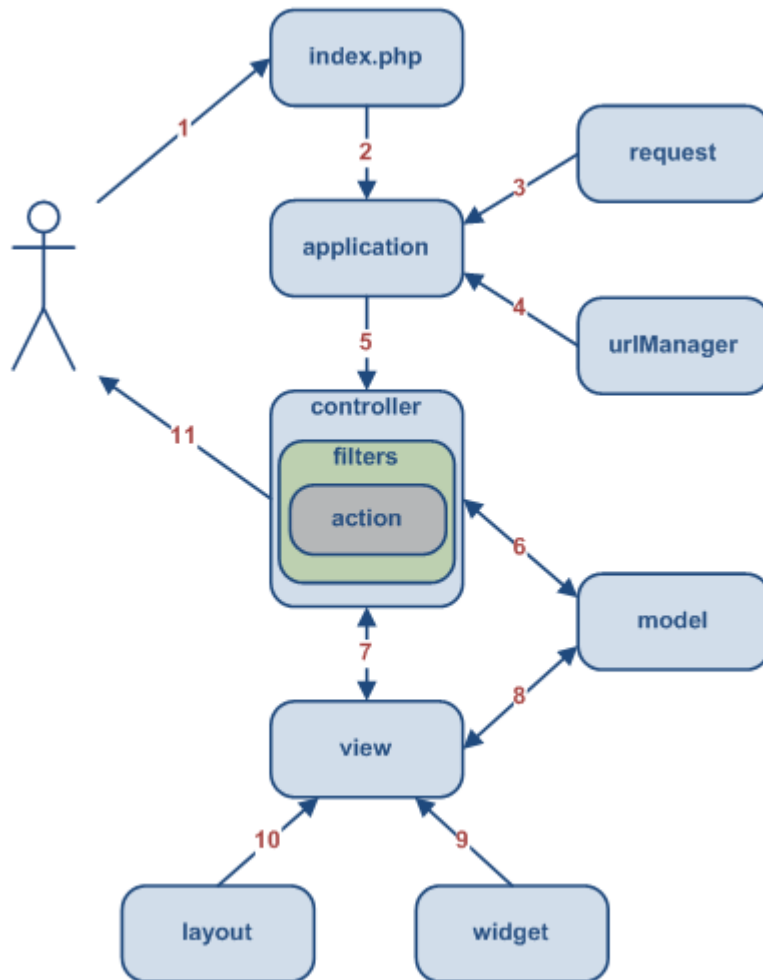
Evento:

“nuevo logro”

→ notifica a otros

2.6 MVC (Modelo Vista Controlador)





TEORÍA

Divide el sistema en:

- Modelo (datos)
- Vista (interfaz)
- Controlador (lógica)

ONG

- Vista → cursos
- Modelo → BD
- Controlador → inscripción